

Grow and Refine: Co-Evolving Memory for
Long-Horizon Machine-Learning Engineering Agents

Qixin Xu

Machine-Learning Engineering

- Long Horizon:

- Many turns
- each turn provide informative feedback

→ How to utilize these feedback?

- Bag of tricks:

- Human >> Sota Agent
- Human know rich specific tricks, agent know limited general method

→ How to close the gap?

Motivation: Memory should be a flywheel, not a pile

A. How does the agent **use** memory?【字太多，分析之前讲出task的特异性，这个特异性决定问题的回答】

a. **trajectory-level:**

- i. Inject all relevant memory items as a single block at the start of the trajectory
- ii. Static across turns, agent can't reconsider per action

b. **turn-level:**

- i. Re-retrieve memory per turn, conditioned on the agent's current action
- ii. agent re-judges strategy each turn

B. How does the agent **update** memory?

a. **append-only:**

- i. Add each new lesson as a new bullet
- ii. Unbounded growth + accumulating contradictions

b. **rewrite-only:**

- i. Modify existing entries in place
- ii. Loses past wisdom, no traceability

c. **dual-update:**

- i. memory **grows** and **self-corrects**
- ii. Delta-append: grow with new validated insights
- iii. Caveat-update: attach scoped warnings to existing entries that were over-generalized

Use memory: Per-turn Action-Aware Retrieval

- Three Agent actions: draft, improve, debug 【可以去掉三个点, 指出这里是agent的action】
 - Draft: propose a new candidate solution from scratch
 - Improve: refine the current best solution
 - Debug: fix a broken solution
- How action shapes memory **writes**
 - a valid Draft, a score-improving Improve, or a successful Debug
 - Memory grows only with **task-relevant evidence**【口头解释是什么】; failed exploration doesn't pollute the pool
- How action shapes memory **reads** 【三个合成一点, 因为都是action-aware】
【draft和improve可能是correlated的, 不区分3个action的效果可以看下】
 - A Draft turn retrieves only past successful Drafts
 - An Improve turn retrieves only past Improves
 - A Debug turn retrieves only past Debug fixes
- Retrieval: Semantic similarity ranking with text embedding【可以不写, 没有特殊性】

Update memory: Dual Memory Update through flywheel

After round t (Agent + memory v_t):

Memory update:

- Growth operator Δ 【朴素语言解释】【小的bullet point字太多】
 - Agent scans round- t trajectories
 - Selects turns: a valid Draft, a score-improving Improve, or a successful Debug
 - Agent summarizes the insights and each becomes a new memory entry

→ memory **covers more strategy space**
- Scope operator caveat 【朴素语言解释】【小的bullet point字太多】
 - Identify tasks that regressed (Agent + memory $v_t < \text{Agent}$)
 - For each regression, trace **which retrieved entry's insight the agent actually applied**
 - Attach a structured scope caveats to that entry

→ memory **refines the boundary** of when each insight is trustworthy

Return memory v_{t+1} and enter round $t+1$

Memory Enables Group Continual Learning【标题改一下, mutli-agents, co-evolving】

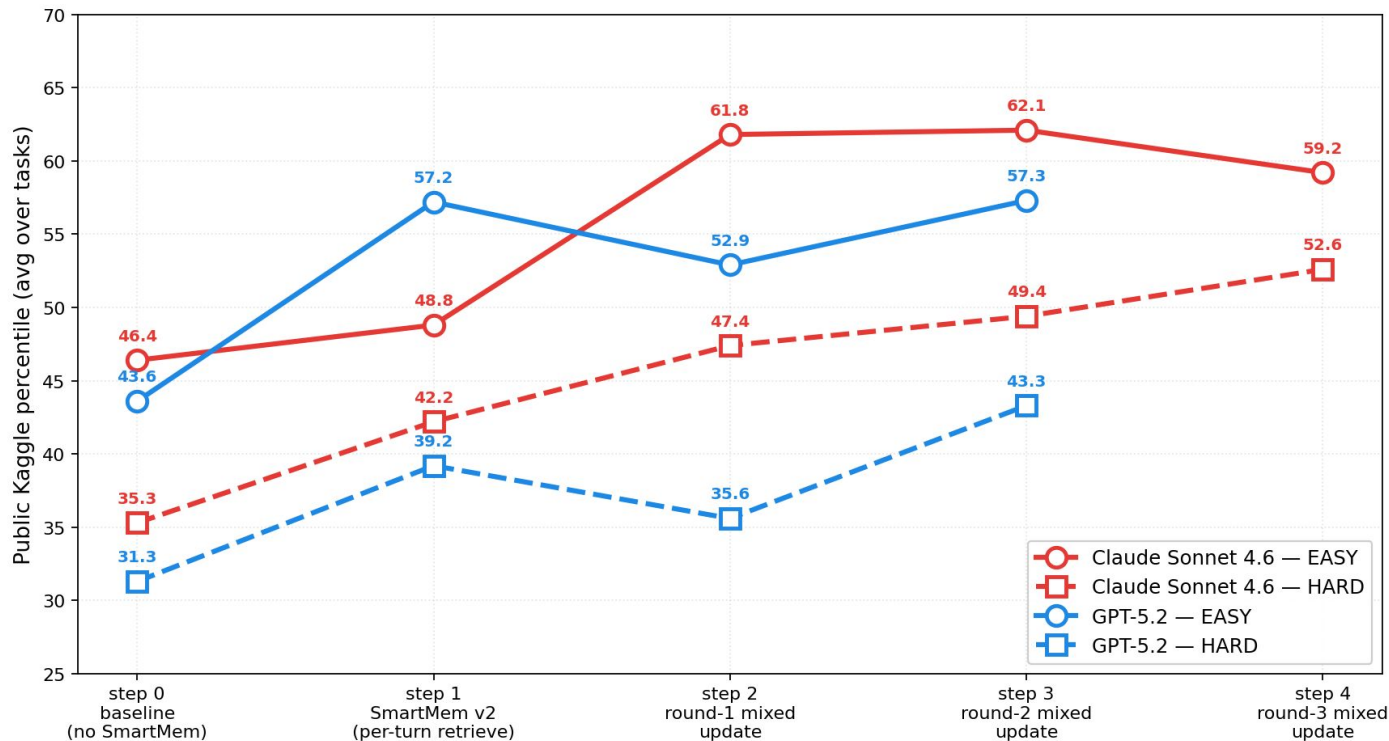
- Shared memory, heterogeneous writers 【三个小点不用列举】
 - A memory pool is mounted by every agent
 - retrieval is model-agnostic
 - Insights from different model families are complementary, not redundant
- Group flywheel across rounds 【flywheel -> continue learning】【每个bullet point concise一点】
 - Read step (per round): all agents query the shared memory v_t
 - Write step (after round):
 - every agent's successful trajectories contribute new delta entries;
 - every agent's regressions trigger caveats
 - both attached to the same pool
 - Memory v_{t+1} is the union of all agents' updates
 - one model's mistake becomes everyone's caveat;
 - one model's success becomes everyone's insight

Results: Setup

- **Benchmark:** 18 real Kaggle competitions from **DSGym**
- **Metric:** Kaggle public leaderboard percentile
- **Backbones:** claude sonnet 4.6, GPT-5.2, Gemini 3 flash
- **Baselines and Ablations:**
 - static memory, no flywheel
 - Δ -only update
 - Caveat-only update
 - Trajectory-level memory
 - Single-model vs. multi-model
 - Update on the fly
 - Other baseline, ACE
 - Different retrieval method

Dual Update【main results, 用表格, 动态变化用折线】

Mixed-update flywheel: cross-model, cross-split progression
(both backbones, both splits, only the 'mixed update' ablation)



Dual update vs. Δ -only update / Caveat update【三个折线】

Per-turn memory vs. trajectory-level memory【表格】

Group wisdom vs. single model knowledge【表格】